实验物理中的统计方法 作业5

Updated 2026-04-13 17:46 GMT+8
*Compiled by 徐前 物理学院 (2026 Spring)

4.3

解：
本底事件期待值$\nu_b = 3.9$，则在无新信号的假设下，发生事件n的概率为

$$P(n|H_0) = \frac{\nu_b^n}{n!} e^{-\nu_b}$$

计算p值，

$$p = \sum_{n=16}^{\infty} \frac{\nu_b^n}{n!} e^{-\nu_b} = 3.58 \times 10^{-6}$$

其中利用关系式

$$\sum_{n=0}^{m} \frac{\nu^n}{n!} \mathrm{e}^{-\nu} = 1 - F_{\chi^2}(2\nu; n_d = 2(m+1))$$

代码：

```
nu = 3.9
m = 15
ndof = 2 * (m + 1)
# 在泊松与卡方转换的恒等式中，chi2 的值对应 2*nu
chi2_val = 2.0 * nu
p_value = ROOT.TMath.Prob(chi2_val, ndof)
```

4.4

代码：

```
import ROOT
import array

def main():
    # ==========================================
    # 0. 准备数据
    # ==========================================
    bins = 20
```

```python
    edges = [0.5 * i for i in range(21)]  # [0.0, 0.5, 1.0, ..., 10.0]
    # 将 Python list 转换为 ROOT 可以接受的 array 格式
    edges_arr = array.array('d', edges)

    n_data = [1, 0, 3, 4, 6, 3, 3, 4, 5, 7, 4, 5, 2, 0, 1, 0, 0, 1, 0, 0]
    nu_th1 = [0.2, 1.2, 1.9, 3.2, 4.0, 4.5, 4.7, 4.8, 4.8, 4.5, 4.1, 3.5, 3.0,
2.4, 1.6, 0.9, 0.5, 0.3, 0.2, 0.1]
    nu_th2 = [0.2, 0.7, 1.1, 1.6, 1.9, 2.2, 2.7, 3.3, 3.6, 3.9, 4.0, 4.0, 3.9,
3.5, 3.2, 2.8, 2.2, 1.5, 1.0, 0.5]

    # ==========================================
    # 定义计算卡方统计量的函数
    # ==========================================
    def calc_chi2(obs, exp):
        """计算 Pearson 卡方统计量 sum((n - v)^2 / v)"""
        chi2 = 0.0
        for o, e in zip(obs, exp):
            if e > 0:
                chi2 += (o - e)**2 / e
        return chi2

    # 计算真实数据的 chi2
    chi2_data_th1 = calc_chi2(n_data, nu_th1)
    chi2_data_th2 = calc_chi2(n_data, nu_th2)

    print(f"--- (a) 实验数据的 Chi2 计算结果 ---")
    print(f"Data vs Theory 1: chi2 = {chi2_data_th1:.2f}")
    print(f"Data vs Theory 2: chi2 = {chi2_data_th2:.2f}")
    print("-" * 40)

    # ==========================================
    # (a) 绘图：将观测数据与两种理论画在同一张图上
    # ==========================================
    canvas = ROOT.TCanvas("c1", "Data and Theories", 800, 600)  # type: ignore

    h_data = ROOT.TH1D("h_data", "Experimental Data vs Theories;x;Events",
bins, edges_arr)  # type: ignore
    h_th1 = ROOT.TH1D("h_th1", "Theory 1", bins, edges_arr)  # type: ignore
    h_th2 = ROOT.TH1D("h_th2", "Theory 2", bins, edges_arr)  # type: ignore

    # 填充直方图（注意 ROOT 的 bin 索引从 1 开始）
    for i in range(bins):
        h_data.SetBinContent(i + 1, n_data[i])
        h_th1.SetBinContent(i + 1, nu_th1[i])
        h_th2.SetBinContent(i + 1, nu_th2[i])
```

```
    # 设置样式
    h_data.SetLineColor(ROOT.kBlack) # type: ignore
    h_data.SetLineWidth(2)
    h_data.SetLineStyle(2)           # 将 Data 设为虚线

    h_th1.SetLineColor(ROOT.kRed)    # type: ignore
    h_th1.SetLineWidth(2)
    h_th2.SetLineColor(ROOT.kBlue)   # type: ignore
    h_th2.SetLineWidth(2)

    # 绘制
    # 【新增细节】提取最大值设置Y轴高度，防止不同直方图叠加时超出画幅
    max_y = max([h_data.GetMaximum(), h_th1.GetMaximum(), h_th2.GetMaximum()])
    h_data.SetMaximum(max_y * 1.2)

    h_data.SetStats(0)               # 关闭统计框
    h_data.Draw("HIST")
    h_th1.Draw("HIST SAME")
    h_th2.Draw("HIST SAME")

    # 添加图例
    legend = ROOT.TLegend(0.65, 0.7, 0.88, 0.88) # type: ignore
    legend.AddEntry(h_data, "Data", "l")
    legend.AddEntry(h_th1, "Theory 1", "l")
    legend.AddEntry(h_th2, "Theory 2", "l")
    legend.Draw()

    canvas.Update()
    canvas.SaveAs("4.4_a.png")
    # ==========================================
    # (b) Toy Monte Carlo: 求真实的 Chi2 分布与 P-value
    # ==========================================
    n_toys = 100000  # 生成 10 万次伪实验
    ndf = bins       # 自由度为 20（没有拟合自由参数）

    # 用来存放生成的 Toy Chi2
    h_chi2_toy_th1 = ROOT.TH1D("h_chi2_toy_th1", "True Chi2 Distribution
(Theory 1);#chi^{2};Entries", 100, 0, 50)  # type: ignore
    h_chi2_toy_th2 = ROOT.TH1D("h_chi2_toy_th2", "True Chi2 Distribution
(Theory 2);#chi^{2};Entries", 100, 0, 50)  # type: ignore

    count_th1_greater = 0
    count_th2_greater = 0

    ROOT.gRandom.SetSeed(42)  # type: ignore # 设置随机数种子，保证结果可重复
```

```
    for i in range(n_toys):
        # 生成 Theory 1 的 Toy Data 并计算 chi2
        toy_data_th1 = [ROOT.gRandom.Poisson(e) for e in nu_th1] # type:
ignore
        toy_chi2_th1 = calc_chi2(toy_data_th1, nu_th1)
        h_chi2_toy_th1.Fill(toy_chi2_th1)
        if toy_chi2_th1 >= chi2_data_th1:
            count_th1_greater += 1

        # 生成 Theory 2 的 Toy Data 并计算 chi2
        toy_data_th2 = [ROOT.gRandom.Poisson(e) for e in nu_th2] # type:
ignore
        toy_chi2_th2 = calc_chi2(toy_data_th2, nu_th2)
        h_chi2_toy_th2.Fill(toy_chi2_th2)
        if toy_chi2_th2 >= chi2_data_th2:
            count_th2_greater += 1

    # 计算真实 P-value
    true_p_th1 = count_th1_greater / n_toys
    true_p_th2 = count_th2_greater / n_toys

    # 计算标准的渐进 P-value
    # TMath::Prob 计算的是理论卡方分布的上尾概率 (p-value)
    asymp_p_th1 = ROOT.TMath.Prob(chi2_data_th1, ndf) # type: ignore
    asymp_p_th2 = ROOT.TMath.Prob(chi2_data_th2, ndf) # type: ignore

    print(f"--- (b) 统计检验与 P-value ---")
    print(f"[ Theory 1 ]")
    print(f"  正常的渐进 P-value (Normal Chi2) = {asymp_p_th1:.4f}")
    print(f"  真实的 P-value (Toy MC)          = {true_p_th1:.4f}")
    print(f"[ Theory 2 ]")
    print(f"  正常的渐进 P-value (Normal Chi2) = {asymp_p_th2:.4f}")
    print(f"  真实的 P-value (Toy MC)          = {true_p_th2:.4f}")


if __name__ == "__main__":
    main()
```

(a)

结果:
``

```
Theory 1: chi2 = 15.82
Theory 2: chi2 = 35.97
```

直方图:

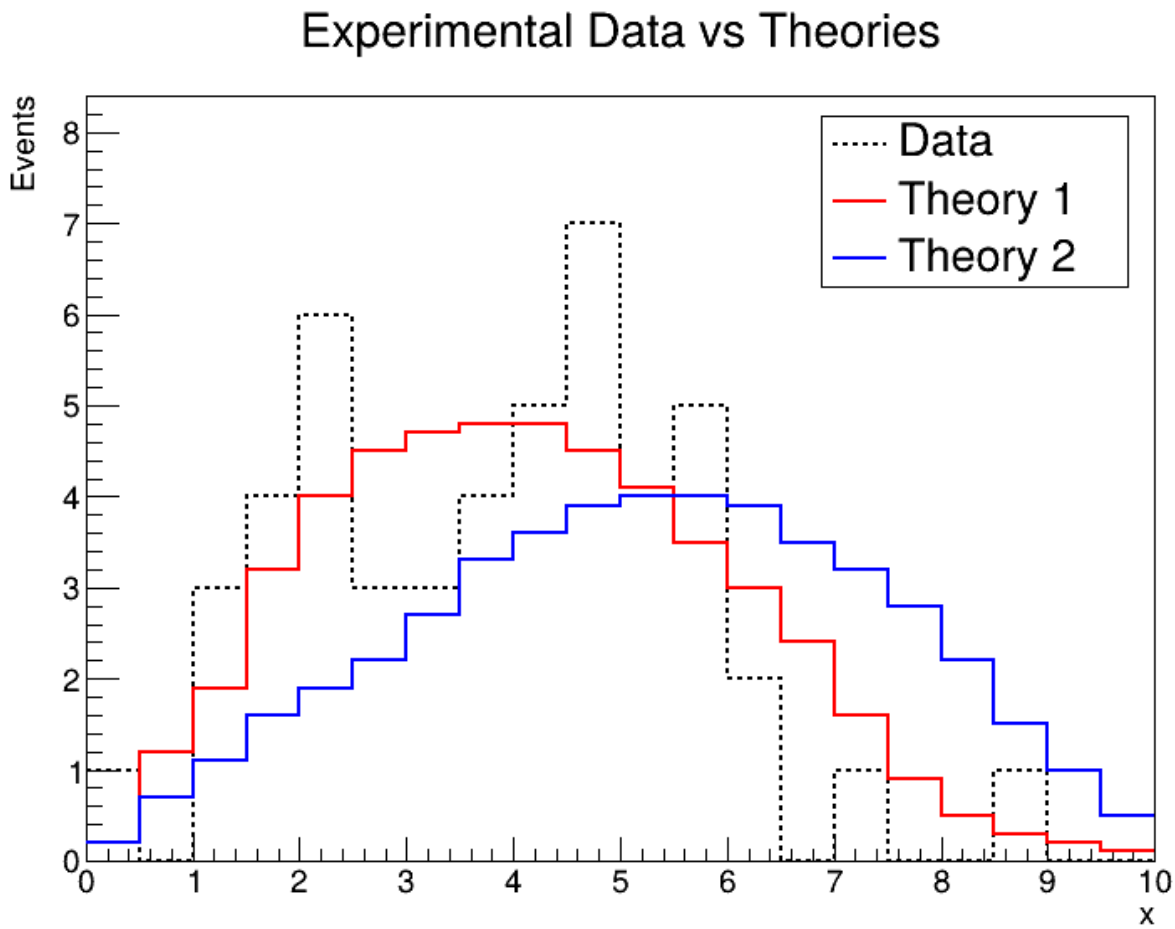

Figure 4.4(a)

(b)

代码:

```
# (b) 附加绘图：真实 Chi2 分布 vs 正常 Chi2 分布
    # ==========================================
    # 创建第二个画布，按 1x2 分割，左边画 Theory 1，右边画 Theory 2
    canvas2 = ROOT.TCanvas("c2", "True vs Normal Chi2 Distributions", 1200,
600) # type: ignore
    canvas2.Divide(2, 1)

    # 定义理论上“正常的卡方分布”概率密度函数 (PDF)
    # [0] 代表我们要传入的第一个参数（即自由度 ndf）
    f_chi2 = ROOT.TF1("f_chi2", "ROOT::Math::chisquared_pdf(x, [0])", 0, 80) #
type: ignore
    f_chi2.SetParameter(0, ndf)
    f_chi2.SetLineColor(ROOT.kBlack) # type: ignore
    f_chi2.SetLineStyle(2) # 虚线表示理论分布
    f_chi2.SetLineWidth(2)
```

```
    # --- 绘制 Theory 1 ---
    canvas2.cd(1)
    # 归一化: Scale( 1.0 / (总积分 * Bin宽度) ), 使其变成概率密度
    if h_chi2_toy_th1.Integral() > 0:
        h_chi2_toy_th1.Scale(1.0 / (h_chi2_toy_th1.Integral() *
h_chi2_toy_th1.GetBinWidth(1)))

    max_y1 = max(f_chi2.GetMaximum(), h_chi2_toy_th1.GetMaximum())
    h_chi2_toy_th1.SetMaximum(max_y1 * 1.1)
    h_chi2_toy_th1.SetLineColor(ROOT.kRed) # type: ignore
    h_chi2_toy_th1.SetLineWidth(2)
    h_chi2_toy_th1.SetStats(0)
    h_chi2_toy_th1.SetTitle("Theory 1: True vs Normal #chi^{2};#chi^{2}
Value;Probability Density")
    h_chi2_toy_th1.Draw("HIST")
    f_chi2.Draw("SAME")

    # 画一条竖线标记我们 (a) 问中算出的真实观测 chi2
    line_th1 = ROOT.TLine(chi2_data_th1, 0, chi2_data_th1, max_y1) # type:
ignore
    line_th1.SetLineColor(ROOT.kBlue) # type: ignore
    line_th1.SetLineWidth(2)
    line_th1.Draw("SAME")

    # 图例
    leg1 = ROOT.TLegend(0.45, 0.7, 0.88, 0.88) # type: ignore
    leg1.AddEntry(h_chi2_toy_th1, "True #chi^{2} (Toy MC)", "l")
    leg1.AddEntry(f_chi2, f"Normal #chi^{2} (ndf={ndf})", "l")
    leg1.AddEntry(line_th1, f"Observed #chi^{2} = {chi2_data_th1:.1f}", "l")
    leg1.SetBorderSize(0)
    leg1.Draw()

    # --- 绘制 Theory 2 ---
    canvas2.cd(2)
    if h_chi2_toy_th2.Integral() > 0:
        h_chi2_toy_th2.Scale(1.0 / (h_chi2_toy_th2.Integral() *
h_chi2_toy_th2.GetBinWidth(1)))

    max_y2 = max(f_chi2.GetMaximum(), h_chi2_toy_th2.GetMaximum())
    h_chi2_toy_th2.SetMaximum(max_y2 * 1.1)
    h_chi2_toy_th2.SetLineColor(ROOT.kRed) # type: ignore
    h_chi2_toy_th2.SetLineWidth(2)
    h_chi2_toy_th2.SetStats(0)
    h_chi2_toy_th2.SetTitle("Theory 2: True vs Normal #chi^{2};#chi^{2}
Value;Probability Density")
```

```
    # 因为 Theory 2 的 chi2 可能会很大，我们需要确保 X 轴范围能包住它
    h_chi2_toy_th2.GetXaxis().SetRangeUser(0, max(60.0, chi2_data_th2 * 1.2))
    h_chi2_toy_th2.Draw("HIST")
    f_chi2.Draw("SAME")

    line_th2 = ROOT.TLine(chi2_data_th2, 0, chi2_data_th2, max_y2) # type:
ignore
    line_th2.SetLineColor(ROOT.kBlue) # type: ignore
    line_th2.SetLineWidth(2)
    line_th2.Draw("SAME")

    leg2 = ROOT.TLegend(0.45, 0.7, 0.88, 0.88) # type: ignore
    leg2.AddEntry(h_chi2_toy_th2, "True #chi^{2} (Toy MC)", "l")
    leg2.AddEntry(f_chi2, f"Normal #chi^{2} (ndf={ndf})", "l")
    leg2.AddEntry(line_th2, f"Observed #chi^{2} = {chi2_data_th2:.1f}", "l")
    leg2.SetBorderSize(0)
    leg2.Draw()

    canvas2.Update()
    canvas2.SaveAs("4.4_b.png")
```

结果：

```
[ Theory 1 ]
  正常的渐进 P-value (Normal Chi2) = 0.7278
  真实的 P-value (MC)              = 0.6523
[ Theory 2 ]
  正常的渐进 P-value (Normal Chi2) = 0.0155
  真实的 P-value (MC)              = 0.0349
```

真实分布和正常卡方分布对比图：

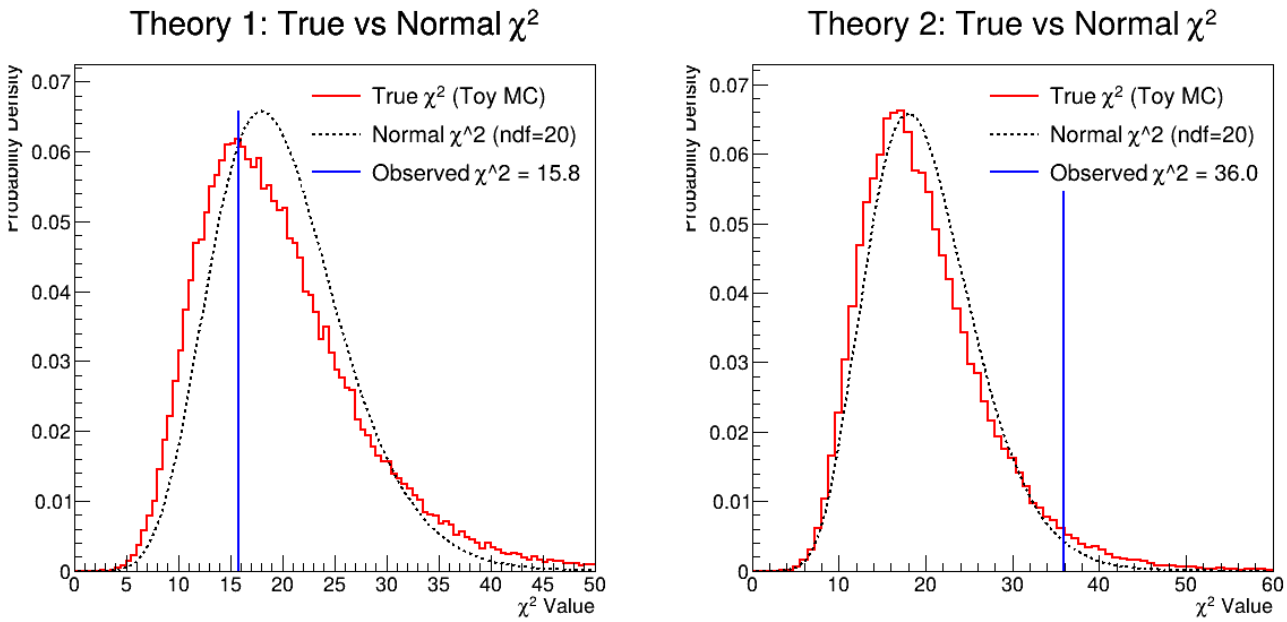

Figure 4.4(b)

运行上述代码后，可以分析得到以下结论：

1. **对于 Theory 1：** 算出的实验卡方较小（数据和模型比较贴合）。发现通过 `TMath.Prob` 算出来的“正常的 P-value”与通过 Toy MC 算出的“真实 P-value”存在差异。由于统计量小，标准的解析公式并不准，必须以 Toy MC 算出来的概率为准。
2. **对于 Theory 2：** 算出来的卡方会很大，意味着数据偏离 Theory 2 的预言非常远。对应的 P-value 会非常小，这意味着我们可以以较高的置信度**拒绝 Theory 2 的假设**。
3. **直方图的可视化：** 图上会呈现出实验黑点分布在红线（Theory 1）周围，而与蓝线（Theory 2）的形状偏差较大，从直观上也印证了上述统计推断的结果。

4.5

(a)

代码:

```
import ROOT
import math

# ==========================================
# (a) 基础数据与公式计算
# ==========================================
m_vals = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
n_obs  = [57, 203, 383, 525, 532, 408, 273, 139, 45, 27, 10, 4, 0, 1, 1]

n_tot = sum(n_obs)
```

```python
# 1. 计算均值 m_bar (Eq 4.7)
sum_m = sum(n * m for n, m in zip(n_obs, m_vals))
m_bar = sum_m / n_tot

# 2. 计算方差 s^2 (Eq 4.8)
sum_variance = sum(n * (m - m_bar)**2 for n, m in zip(n_obs, m_vals))
s2 = sum_variance / (n_tot - 1)

# 3. 计算分散度 t (Eq 4.9)
t_obs = s2 / m_bar

print("=== (a) 样本数据计算 ===")
print(f"总衰变次数 n_tot = {n_tot}")
print(f"样本均值 m_bar   = {m_bar:.4f}")
print(f"样本方差 s^2     = {s2:.4f}")
print(f"分散度 t         = {t_obs:.4f}")
```

结果:

```
=== (a) 样本数据计算 ===
总衰变次数 n_tot = 2608
样本均值 m_bar   = 3.8715
样本方差 s^2     = 3.6962
分散度 t         = 0.9547
```

(b)

注意到 t 值小于1, 为了表征 t 的观测值与泊松假设不相符，应该选取偏离 1 更远的 t, 即需要做双侧检验.

代码:

```python
# (b) 解析 P-value 计算
    # ==========================================
    # 统计量 X2 = (n_tot - 1)*t 服从自由度为 n_tot - 1 的卡方分布
    X2_obs = (n_tot - 1) * t_obs
    ndf = n_tot - 1

    # TMath.Prob 计算的是上尾概率 P(X^2 > X^2_obs)
    p_value_analytical = ROOT.TMath.Prob(2 * ndf - X2_obs, ndf) + 1 -
ROOT.TMath.Prob(X2_obs, ndf)# type: ignore

    print("\n=== (b) 解析 P-value ===")
```

```
    print(f"观测到的 X^2 = {X2_obs:.2f}, 自由度 ndf = {ndf}")
    print(f"解析计算的 P-value = {p_value_analytical:.4f}")
```

结果:

```
观测到的 X^2 = 2488.92, 自由度 ndf = 2607
解析计算的 P-value = 0.1019
```

(c)

代码:

```
# (c) 蒙特卡洛 MC 计算与选作题作图
n_toys = 10000  # 生成一万组伪实验数据
count_extreme = 0

# 画图用的直方图
# 选作题目：记录 (n_tot-1)t，其理论预期是均值为 ndf，方差为 2*ndf 的高斯分布
h_X2_toy = ROOT.TH1D("h_X2_toy", "Toy MC (n_{tot}-1)t Distribution;
(n_{tot}-1)t; Probablity", 100, 2300, 2900) # type: ignore

ROOT.gRandom.SetSeed(42) # type: ignore

print("\n=== (c) 正在运行 Monte Carlo (请稍候...) ===")
for _ in range(n_toys):
    toy_sum_m = 0
    toy_sum_m2 = 0

    for _ in range(n_tot):
        val = ROOT.gRandom.Poisson(m_bar) # type: ignore
        toy_sum_m += val
        toy_sum_m2 += val * val

    toy_m_bar = toy_sum_m / n_tot
    # 快速计算样本方差：(sum(x^2) - n*mean^2)/(n-1)
    toy_s2 = (toy_sum_m2 - n_tot * toy_m_bar**2) / (n_tot - 1)
    toy_t = toy_s2 / toy_m_bar
    toy_X2 = (n_tot - 1) * toy_t

    h_X2_toy.Fill(toy_X2)

    # 计算双侧 P-value（与 1 的偏离程度）
    # 如果模拟出的 t 偏离 1 的距离，大于等于观测到的 t 偏离 1 的距离，则记为极端事件
    if abs(toy_t - 1.0) >= abs(t_obs - 1.0):
        count_extreme += 1
```

```
p_value_mc = count_extreme / n_toys
print(f"Toy MC 计算的双侧 P-value = {p_value_mc:.4f}")

# 绘制结果 (选作题)
canvas = ROOT.TCanvas("c1", "Toy MC vs Gaussian", 800, 600) # type: ignore

# 标准化直方图以便与高斯概率密度函数对比
h_X2_toy.Scale(1.0 / (h_X2_toy.Integral() * h_X2_toy.GetBinWidth(1)))
h_X2_toy.SetLineColor(ROOT.kBlue) # type: ignore
h_X2_toy.SetLineWidth(2)
h_X2_toy.SetStats(0)
h_X2_toy.Draw("HIST")

# 理论上的高斯分布: mean = ndf, variance = 2*ndf, sigma = sqrt(2*ndf)
gauss_mean = ndf
gauss_sigma = math.sqrt(2 * ndf)
f_gauss = ROOT.TF1("f_gauss", "TMath::Gaus(x, [0], [1], 1)", 2300, 2900) #
type: ignore
f_gauss.SetParameters(gauss_mean, gauss_sigma)
f_gauss.SetLineColor(ROOT.kRed) # type: ignore
f_gauss.SetLineWidth(2)
f_gauss.SetLineStyle(2)
f_gauss.Draw("SAME")

# 画一条线表示观测到的值
line_obs = ROOT.TLine(X2_obs, 0, X2_obs, h_X2_toy.GetMaximum()) # type: ignore
line_obs.SetLineColor(ROOT.kBlack) # type: ignore
line_obs.SetLineWidth(2)
line_obs.Draw("SAME")

# 添加图例
legend = ROOT.TLegend(0.6, 0.7, 0.88, 0.88) # type: ignore
legend.AddEntry(h_X2_toy, "Toy MC Data", "l")
legend.AddEntry(f_gauss, f"Gaussian (#mu={gauss_mean:.0f}, #sigma=
{gauss_sigma:.1f})", "l")
legend.AddEntry(line_obs, f"Observed X^{{2}} = {X2_obs:.1f}", "l")
legend.SetBorderSize(0)
legend.Draw()

canvas.Update()
canvas.SaveAs("4.5_c.png")
```

结果:

```
MC 计算的双侧 P-value = 0.0981
```

直方图:

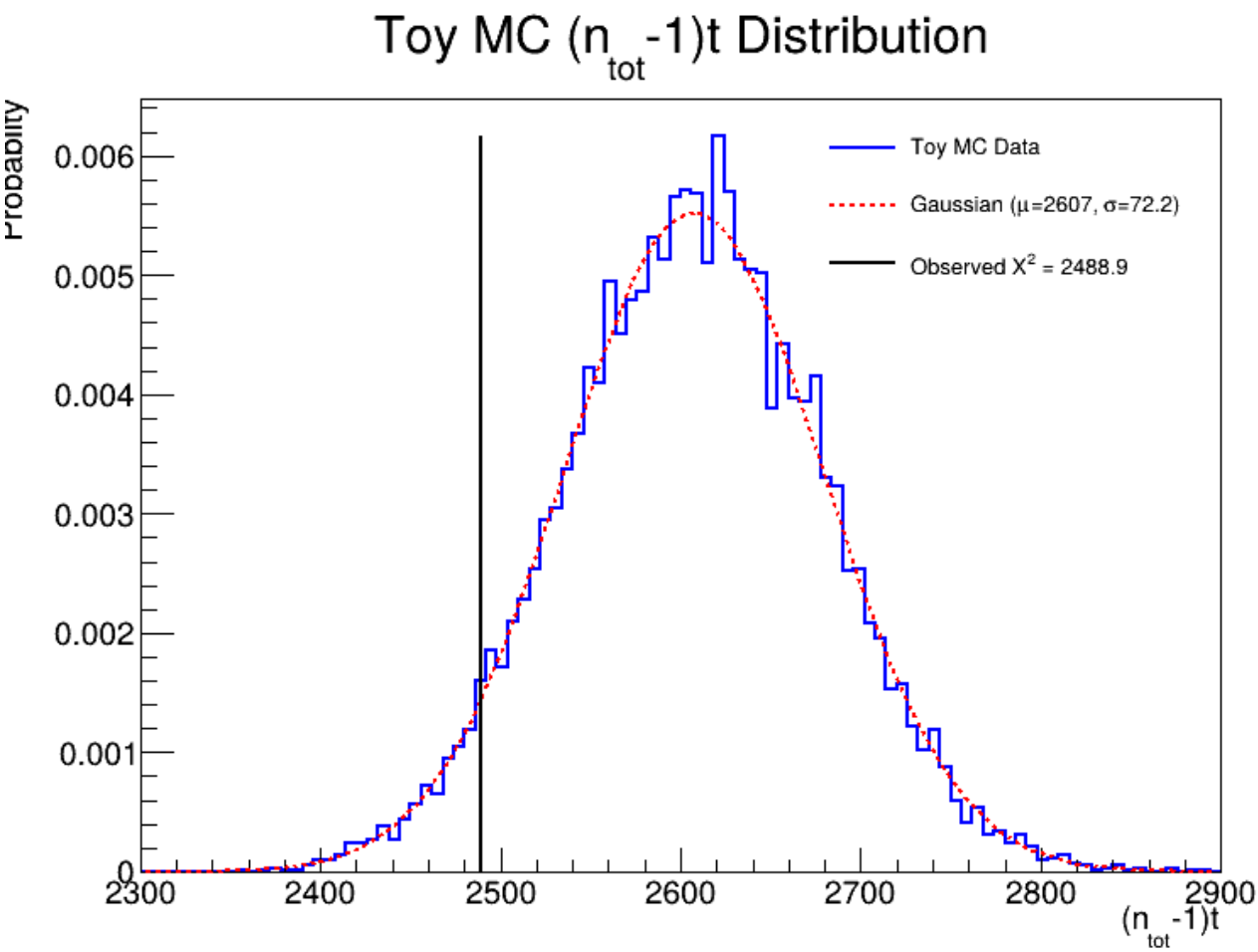

Figure 4.5_c

结论分析

1. **P-value 分析**：代码会算出解析 P-value 和 MC P-value（大约在 0.1 左右）。这个 P-value 远大于 0.05（常规的显著性水平）。
2. **最终物理结论**：既然 P-value 很大，说明**我们不能拒绝“服从泊松分布”的假设**。换言之，观测到的微小偏离完全可以由统计涨落来解释。卢瑟福和盖革的数据**并没有表现出显著的“衰变簇团”效应**，无法证明 α 衰变不是一个相互独立的纯随机过程。